



به نام خدا

طراحی کامپیوتری سیستم‌های دیجیتال

پاییز ۱۴۰۵

پروژه اول : مهندسی پرامپت در طراحی RTL

طراحان : هاتف رضائی - مهدیس میرزایی

هدف پروژه :

در این تمرین با مجموعه‌ای از تکنیک‌های پرامپت‌نویسی آشنا می‌شوید. تکنیک‌هایی که کمک می‌کنند به‌جای صحبت کردن با یک مدل ، از آن به‌شکلی ساختار یافته و درست در توسعه یک سیستم RTL استفاده کنید. اینکه مدل چه خروجی‌ای به شما می‌دهد و کیفیت خروجی به چه صورت است بیش از هر چیز به ورودی وابسته است: به اینکه چه چیزی، با چه دقتی و در چه قالبی به مدل ورودی می‌دهید. پس نوشتن پرامپت خودش بخشی از کار طراحی است. برای هر تکنیک، ابتدا مبانی آن و سپس اثرش بر خروجی مدل در یک طراحی RTL بررسی می‌شود.

پیش از تولید کد، باید مشخصات طراحی را تا حد ممکن صریح و بدون ابهام بنویسید. زبان و استانداردی که به کار می‌برید (برای مثال SystemVerilog با گفتن نسخه‌ای که استفاده می‌کنید)، ابزار شبیه‌سازی، فهرست ماژول‌ها، سنکرون یا آسنکرون بودن سیگنال‌هایی مانند rst، رفتار رجیسترها پس از Reset و محدودیت‌های طراحی و هرچیزی که در این طراحی موثر است, به عبارتی هر چیزی که نگویید، مدل خودش با یک فرض دلخواه پر می‌کند. نتیجه کدی است که شاید بدون خطا کامپایل شود اما آن چیزی نیست که می‌خواستید. این همان چیزی است که به آن hallucination مدل می‌گویند. معماری سیستم را هم باید خودتان و پیش از سراغ رفتن به مدل مشخص کنید و به زیرماژول‌های کوچک با وظیفه‌ی روشن بشکنید. اگر طراحی از دو بخش Controller و Datapath تشکیل شده، مرز میان این دو باید از قبل معلوم باشد

هدف اصلی این تمرین نشان دادن یک نکته است. هرچه مشخصات ورودی دقیق‌تر، معماری روشن‌تر و تولید کد مرحله‌به‌مرحله‌تر باشد، خروجی مدل هم قابل‌کنترل‌تر و قابل‌ارزیابی‌تر می‌شود و بیشتر به یک طراحی واقعی RTL نزدیک است و می‌توان از آن استفاده کرد. در پایان این تمرین می‌دانید مدل‌های زبانی را کجای فرایند طراحی و پیاده‌سازی سخت‌افزار در سطح RTL می‌شود به کار گرفت و چگونه.

در ادامه برای هر روش توضیحات لازم آمده است. اگر خواستید مطالب بیشتری بدانید، می‌توانید به مقاله‌ی مرجع همان بخش مراجعه کنید. انتهای توضیح هر بخش یک تسک عملی آمده که باید طبق فرمت اعلام‌شده تکمیل کنید و تحویل دهید. تحویل خارج از فرمت، امتیاز آن بخش را از بین می‌برد.

بخش ۱ - تثبیت کردن

یکی از مهم‌ترین نکات این است که همه پیش‌فرض‌ها گفته شود؛ مدل هر بخشی از مسئله را که مبهم باشد یا به عبارتی به طور دقیق گفته نشده است، خودش با یک فرض دلخواه تکمیل می‌کند و این می‌تواند کدی تولید کند که از نظر نحوی درست است اما هدف واقعی طراحی را برآورده نمی‌کند. بنابراین باید نسخه دقیق زبان (برای نمونه SystemVerilog-2012) یا blocking یا nonblocking و سایر جزئیات مشابه به‌طور کامل و دقیق در ابتدای کار مشخص شوند.

> **تسک ۱**
>
> فایل زمینهٔ پروژهٔ خود را بنویسید؛ در این فایل، صورت‌مسئله باید به‌طور کامل و بدون ابهام بیان شود و تمام جزئیات لازم ذکر گردد.

فرمت تحویل: `01_context.md`

بخش ۲ - شکستن پروژه

شما باید تصمیم‌گیری درباره تقسیم‌بندی پروژه را خودتان انجام دهید و به مدل بگویید این تقسیم بندی به چه صورت است نه اینکه این کار را به آن واگذار کنید. به عبارتی هرچه درخواستی که از مدل دارید بزرگ‌تر و کلی‌تر باشد، مدل جزئیات کمتری را با دقت پردازش می‌کند. در مقابل، اگر خودتان معماری را به زیرماژول‌های کوچک‌تر بشکنید و از مدل بخواهید هر ماژول را جداگانه تکمیل کند، کیفیت خروجی به‌طور قابل توجهی بالا می‌رود. بهتر است برای هر زیر ماژول یک گفت‌وگوی جداگانه باز کنید و فایل زمینه را در نخستین پیام هر گفت‌وگو تکرار کنید. برای هر ماژول هم باید هدر آن را دقیق

مشخص کنید. یعنی نام سیگنال‌ها، تعداد بیت هر پورت و جهت آن (ورودی یا خروجی) به‌همراه ترتیب پورت‌ها، تا در مرحله‌ی اتصال ماژول‌ها به یکدیگر خطایی پیش نیاید.

> **تسک ۲**
>
> طراحی هدف را به زیرماژول‌ها بشکنید، برای هر زیر ماژول یک module header بنویسید و نموداری ساده از نحوه‌ی اتصال ماژول‌ها به یکدیگر رسم کنید.

فرمت تحویل: `partitioning.md_02` به‌همراه تصویر نمودار در فایلی فشرده با نام `sub_modules.zip`

بخش ۳ - مشخص کردن نماد

در طراحی سخت‌افزار، خطای مدل معمولا از نوشتن نادرست کد نمی‌آید، بلکه از خواندن نادرست نمادها و جدول‌ها می‌آید. مثلاً ممکن است مدل یک جدول درستی را اشتباه بخواند و به‌جای AND عملگر OR را پیاده کند. پس کار خواندن و تفسیر جدول‌ها را به مدل نسپارید و خودتان آن را به طور واضح و صریح به مدل بگویید. برای مثال نوع Reset (سنکرون یا آسنکرون) و مقدار رجیسترها پس از آن، مقدار دیفالت برای حالت‌های پوشش‌داده‌نشده، رفتار ماژول در برابر ورودی نامعتبر و state غیرقابل‌دسترس، عرض بیت و علامت‌دار یا بدون‌علامت بودن سیگنال‌ها. همچنین در طراحی‌های مبتنی بر FSM فهرست کامل state ها به‌همراه شرط هر گذار و خروجی هر state. اگر ماژول موردنظر یک ساختار متعارف و شناخته‌شده است (مانند Carry-Lookahead Adder یا FIFO)، به‌جای توصیف رفتاری از صفر، نام استاندارد آن را بیاورید.

> **تسک ۳**
>
> برای هر یک از زیرماژول‌ها یک spec کامل بنویسید. اگر ماژول جدول دارد، آن جدول را به طور کامل توضیح دهید به عبارتی بگویید در چه ثانیه بر حسب چه ورودی خاصی چه خروجی‌ای باید باشد.

فرمت تحویل: `spec.md_03`

بخش ۴ - ساختاردهی

وقتی تمام اطلاعات را در قالب یک پاراگراف پیوسته به مدل می‌دهید، مدل خودش باید حدس بزند کدام جمله دستور است، کدام داده، کدام اولویت بیشتری دارد و حتی خروجی در چه قالبی باید باشد و اگر هر یک از موارد گفته شده به طور کامل مشخص نباشد هر یک باعث خطا می‌شود. برای این کار پرامپت را به شش بخش تقسیم کنید:

- **Role** : زاویه‌ی دید و هدفی که مدل باید مسئله را از آن ببیند. عبارت‌های کلی مانند "**تو یک متخصص هستی**" اثری بر خروجی ندارند چون دقیقا هدف متخصص بودن مشخص نشده است اما جمله‌ای مانند "**تو یک طراح RTL هستی که کد قابل‌سنتز در Verilog-2005 می‌نویسی**" مشخص می‌کند مدل باید به چه نکاتی حساس باشد.
- **Context** : همان فایل زمینه‌ی بخش اول است.
- **Input** : این بخش ورودی را بین علامت " " قرار دهید تا مدل متوجه شود این قسمت داده است، نه دستور. اگر این کار را نکنید، ممکن است جمله‌ای از داخل spec را دستور جدید تلقی کند که اشتباه است.
- **Instructions** : فهرستی شماره‌گذاری‌شده و مرتب از کارهایی که باید انجام شود. اگر چند کار را در یک جمله بگویید معمولا مدل یکی از آن‌ها را ناقص انجام می‌دهد یا اصلا انجام نمی‌دهد.
- **Constraints** : محدودیت‌های صریح. مانند الزام به استفاده از ساختارهای قابل‌سنتز، یا این که مدل اگر به ابهامی خورد به‌جای حدس زدن آن را اعلام کند و به طور پیش‌فرض از چیزی استفاده نکند.
- **Output Format** : قالب دقیق خروجی مورد انتظار. این بند کمترین هزینه و بیشترین اثر را دارد معمولا نادیده گرفته می‌شود ولی بسیار مهم است و باید گفته شود.

همچنین بهتر است ساختارهای شرطی یک ماژول به صورت if else نوشته شود تا به صورت توصیفی.

تسک ۴

ابتدا یک پرامپت ساده و بدون ساختار برای زیرماژول‌ها بنویسید و اجرا کنید. سپس همان درخواست را طبق قالب معرفی‌شده در این بخش بازنویسی و دوباره اجرا کنید. هر دو خروجی را ارائه دهید و تفاوت‌هایشان را تحلیل کنید.

فرمت تحویل : `prompt_before_after.md_04`

بخش ۵- برنامه‌ریزی :

وقتی خودتان می‌خواهید یک مسئله‌ی طراحی را حل کنید، مستقیم سراغ نوشتن کد نمی‌روید. برای مثال بررسی می‌کنید به چه صورت باید طراحی شود و چه رجیسترهای لازم است. مدل‌های زبانی معمولا این مرحله را ندارند و از همان اول کد تولید می‌کنند. مشکل این است این رویکرد این است که تصمیم‌های طراحی داخل کد پنهان می‌مانند و اگر باگی پیش بیاید، فهمیدن اینکه خطا از منطق طراحی است یا از پیاده‌سازی آن سخت می‌شود. برای رفع این مشکل، مدل را وادار کنید پیش از نوشتن هر خط کد یک پلن پیاده‌سازی بدهد: چه رجیسترها و پارامترهایی لازم است، چند بلوک always می‌خواهد و هر کدام چه رفتاری را پیاده می‌کند، stateهای FSM و شرط هر transition چه چیزی است و به صورت blocking خواهند بود یا non-blocking. اگر پلن را مناسب دیدید، آن‌وقت از مدل بخواهید کد نهایی را تولید کند و کنار هر بخش از کد، کامنتی با شماره‌ی step مرتبط با همان گام در پلن بگذارد. جدا کردن مرحله‌ی استدلال از مرحله‌ی کدنویسی دو مزیت دارد. اول اینکه کیفیت استدلال مدل بالا می‌رود. دوم اینکه در مرحله‌ی بازبینی می‌توانید کد را خط‌به‌خط با پلن تطبیق دهید و مطمئن شوید مدل به درستی عمل کرده است یا خیر.

تسک ۵

برای هر زیر ماژول ابتدا پلن پیاده‌سازی و بعد کد نهایی به‌همراه کامنت‌های شماره‌ی step.
همچنین در گزارش اعلام کنید آیا طبق پلن خروجی داده شده است یا خیر

فرمت تحویل: `plan.md_05` و `code.v_05`

بخش ۶ - پرامپت‌نویسی در چندمرحله

در یک تیم واقعی طراحی سخت‌افزار، یک نفر مسئول تمام مراحل نیست. یکی طرح را تدوین می‌کند، یکی کد می‌زند، یکی بازبینی می‌کند و یکی تست می‌گیرد. دلیل این تفکیک نقش‌ها این است که پیدا کردن و اصلاح خطای کد خود سخت‌تر از ارزیابی کد دیگری است.

مدل‌های زبانی هم به طور مشابه هستند. اگر لاگ خطا را به همان گفت‌وگویی بدهید که کد را تولید کرده، مدل معمولا همان اشتباه را تکرار می‌کند. پس نباید صرفا در یک گفت و گو با آن صحبت کرد و می‌توان این فرایند را به ۴ قسمت تقسیم کرد.

- **Planner**: spec را به گام‌های ریز اجرایی می‌شکند.
- **Programmer** : کد را دقیقا طبق همان گام‌ها (در مرحله قبلی مشخص شده) و به همراه کامنت می‌نویسد.
- **Reviewer** : کد را با پلن تطبیق می‌دهد و گام‌های جا افتاده یا تغییریافته را مشخص می‌کند.
- **Evaluator:** کد بازبینی شده را با استفاده از دستور هایی که مدل داده است اجرا کنید و لاگ خطاها را دوباره به مدل بدهید، در نتیجه مدل می تواند انحراف از مدل کاملا درست را تشخیص داده و با استفاده از داده های درون context اش، اصلاحات مورد نیاز را اعلام کند.

توجه کنید که این فرایند خطی و پایپ‌لاین‌گونه است. صورت پروژه به مدل اول داده می‌شود، خروجی آن ورودی مدل بعدی می‌شود و همین‌طور تا انتهای زنجیره ادامه پیدا می‌کند. پس باید بر خروجی هر مرحله اشراف کامل داشته باشید و پیش از فرستادن آن به مرحله‌ی بعد بازبینی‌اش کنید. هر خطایی که در یک مرحله از دست برود، تا انتهای پایپ‌لاین منتقل می‌شود و رفعش در مراحل بعدی سخت‌تر خواهد بود.

تسک ۶

پلن را به‌عنوان ورودی به Programmer بدهید و کد متناظر را بگیرید. خروجی را به Reviewer بدهید تا با پلن تطبیقش دهد. سپس کد بازبینی‌شده را با testbench اجرا کنید و لاگ خطا را به Evaluator برگردانید. این چرخه را تا پاس شدن کامل تست تکرار کنید.

فرمت تحویل:

پرامپت خروجی : `06_1_planner.md`

کد تولید شده : `06_2_programmer.v`

گزارش تطبیق کد با پلن : `06_3_reviewer.md`

لاگ خطا و تحلیل مدل از علت آن : `06_4_evaluator.md`

سایر نکات

- انجام این تمرین به صورت گروه های دونفره خواهد بود
- فایل ها و گزارش خود را تا قبل از موعد تحویل، با نام CAD_CA1_<SID1>_<SID2>.zip در محل مربوطه در صفحه درس آپلود کنید.
- توجه کنید که باید به کد های ارائه داده شده تسلط داشته باشید، تولید شدن کد ها توسط llm ها توجیه عدم تسلط نمی‌باشد. به عبارتی به ازای هر بخش باید تسلط کامل بر هر قسمتی از کدها داشته باشید.
- دقت کنید پروژه باید به صورت مرحله به مرحله انجام شود و در چند پرامپت انجام شود. (برای مثال در بخش ۶ توضیحات کامل‌تر داده شده است)
- ماژولاریتی و تمیزی کدها بخشی از نمره ی پروژه را در بر دارد
- در صورت هرگونه ابهام می توانید از طریق ایمیل با طراحان در ارتباط باشید.

- هدف این تمرین یادگیری شماست! در صورت کشف تقلب، مطابق با قوانین درس برخورد خواهد شد.

موفق باشید